Your monitoring tool is holding your company hostage.

dd2dash0 makes leaving easy.

Who We Are



Henry Li

Pitch and product

PPE @ LSE

Co-founder @ A* AI (VC-backed, 10K+ users)



Amir Gulubayli

Engineering lead

Maths @ UCL

Winner @ Google AI hackathon



Ali Kadhim

Query translator

Aeronautics @ Imperial

Founder @ Tutevia



Michael Fleischmann

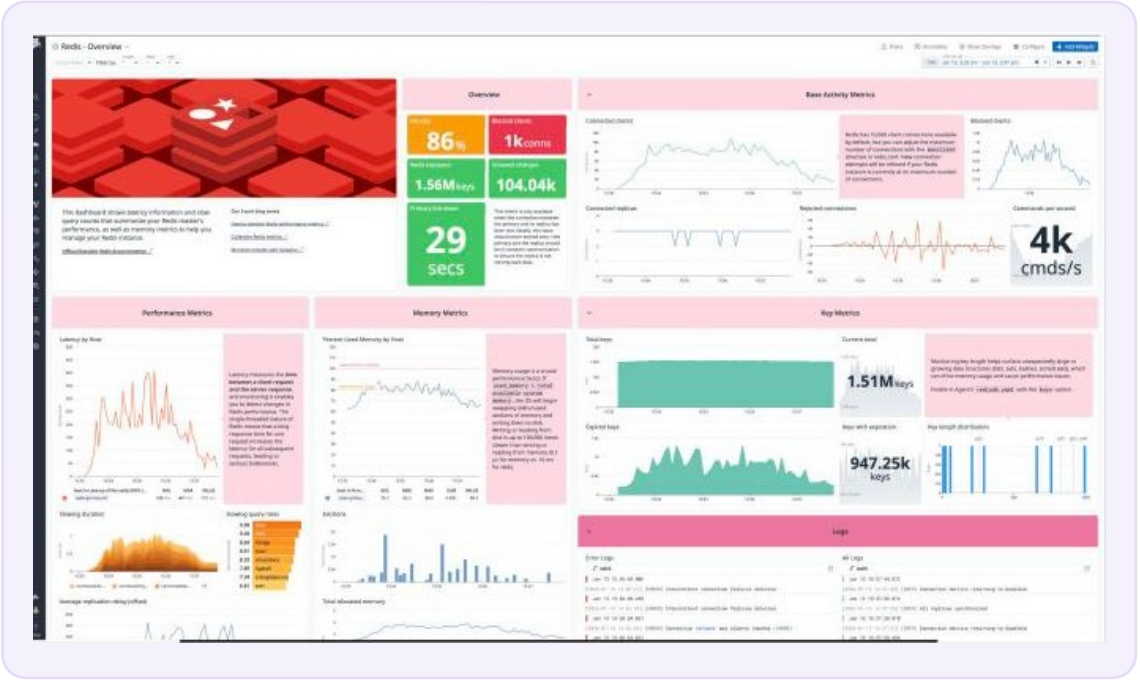
Dash0 import

Elec. & Computer Eng. @ TUM

AI migration research @ Goethe University

Built in 24 hours. Query parser, 200 written rules, Dash0 importer, over 700 tests.

The Problem



Datadog's Redis dashboard, one of many

- Rebuilding dashboards and alarms by hand takes **weeks**.
- Get one alarm wrong, and you miss the incident it was meant to catch.
- dd2dash0 translates Datadog's 516 public dashboards in **under 5 seconds**.

Source: Dash0's own challenge brief says teams spend weeks or months rebuilding by hand. Under 5 seconds is translation only. A person still reviews.

Our Solution: dd2dash0

It translates Datadog dashboards and alarms with written rules. No AI does the translating.

1

Plan

Translates everything. Imports nothing.

2

Review

A person checks each item and its label.

3

Apply

Imports only what was approved.

4

Watch

Alarms start switched off.

Every widget gets a written rule, or goes to a person with the reason.

Same input, same output, every run.

How We Check It

Live, against Datadog. One test counter, logs sent at known times, and a live Redis host.

36 of 36

minutes identical

The same counter, sent to Datadog and Dash0 at the same moment

Exact

log counts

Up to 5,660 logs sent at known times. Every minute counted exactly

0.004%

apart on memory

Live Redis feeding both tools. The alarm test is on the Proof slide

700+

automated tests, run on every build

Syntax

Every query parsed with Prometheus's own grammar

Mappings

Every mapping checked against OpenTelemetry and Datadog sources

Schema

Every payload checked against Dash0's API, then read back

Maths

Written proofs with error terms, and 4 reviewed example dashboards

Demo

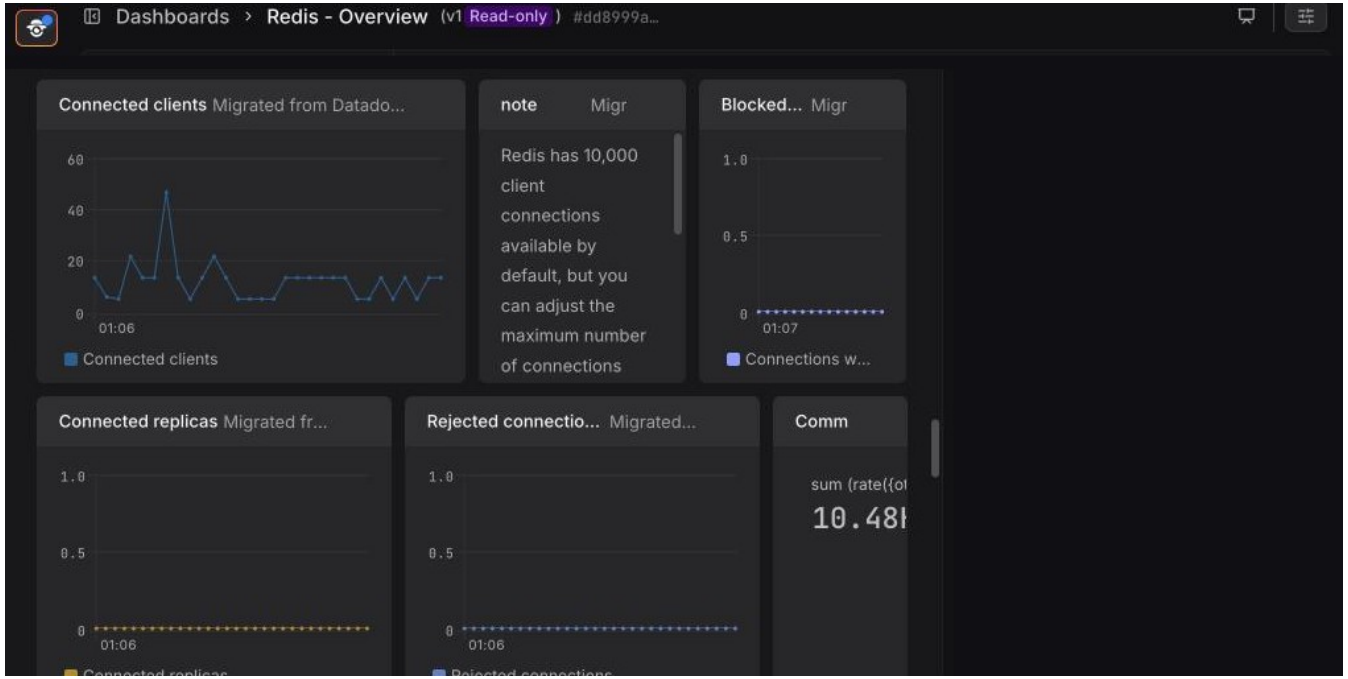
Datadog's Redis dashboard in Dash0: 36 widgets, 0 errors.

21 with live data

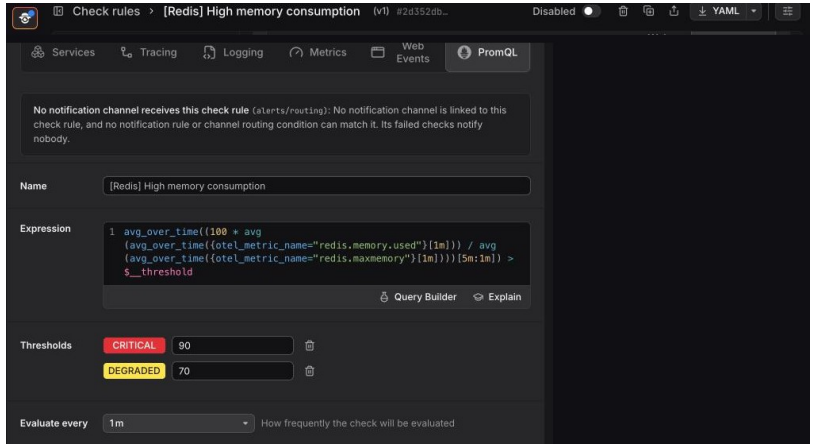
7 notes

2 log panels

6 sent to a person, with the reason



The migrated dashboard, live in Dash0



The memory alarm as a Dash0 check rule. It arrives switched off.

Results

85.6%

of panels labelled exact under a stated condition

12,234 of 14,285 panels



93.7%

of distinct queries and monitors translated (13,286 of 14,172)

57.4%

of alerts translated (189 of 329), out of about 65% possible

Source: Datadog's own public library (DataDog/integrations-core), 516 dashboards, 14,285 panels, 329 monitors. The 85.6% includes 1,736 text notes copied as written. Without them, 83.7%. Translated, not imported.

Proof

13 of 14

Redis panels within 1% of Datadog

Memory, same host, same 7 minutes



Fired

our migrated alarm, in Dash0

- Switched on for the test
- Redis memory went past 90%
- Datadog alerted, and our rule fired
- Then it resolved

No channel was linked, so nobody got paged.

Source: team spot check on an earlier build. One Redis host feeding both tools, 7-minute window, read-only comparison script. Full numbers in the backup.

Limits

57.4% of alerts translate

Out of about 65% possible without guessing. Anomaly monitors stay with a person.



3 integrations mapped

Redis, Postgres and NGINX. Others show no live data until they are mapped.

1 Redis host tested live

A customer's setup gets its own comparison before any alarm goes live.

Not enterprise-grade yet

No secret store, access control or audit trail. Notification channels aren't migrated.

Our Ask

Import from Datadog

Proposed for Dash0, powered by dd2dash0. Free and open source.

Give every Datadog user a way out.

Backup: The Labels

Every panel gets one of 4 labels. Counts are across all 14,285 panels in Datadog's public library.

Label	Panels	What it means, with a checked example
Exact under a condition	**12,234**	Matches exactly while its stated condition holds. Example: log counts, checked with logs sent at known times.
Bounded	**151**	Within a written error term. Example: percentiles, probed on real OpenTelemetry histograms.
Manual	**1,198**	Keeps the original query and the reason. Example: primary link down has no OpenTelemetry metric.
Unsupported	**702**	Dash0 has no equivalent panel, so it becomes a placeholder. Example: key length distribution.

No panel is ever labelled plain "exact". Every label points to a named rule in docs/rules.md, with its assumptions. Exact under a condition includes 1,736 text notes, copied as written.

Backup: Redis, Panel by Panel

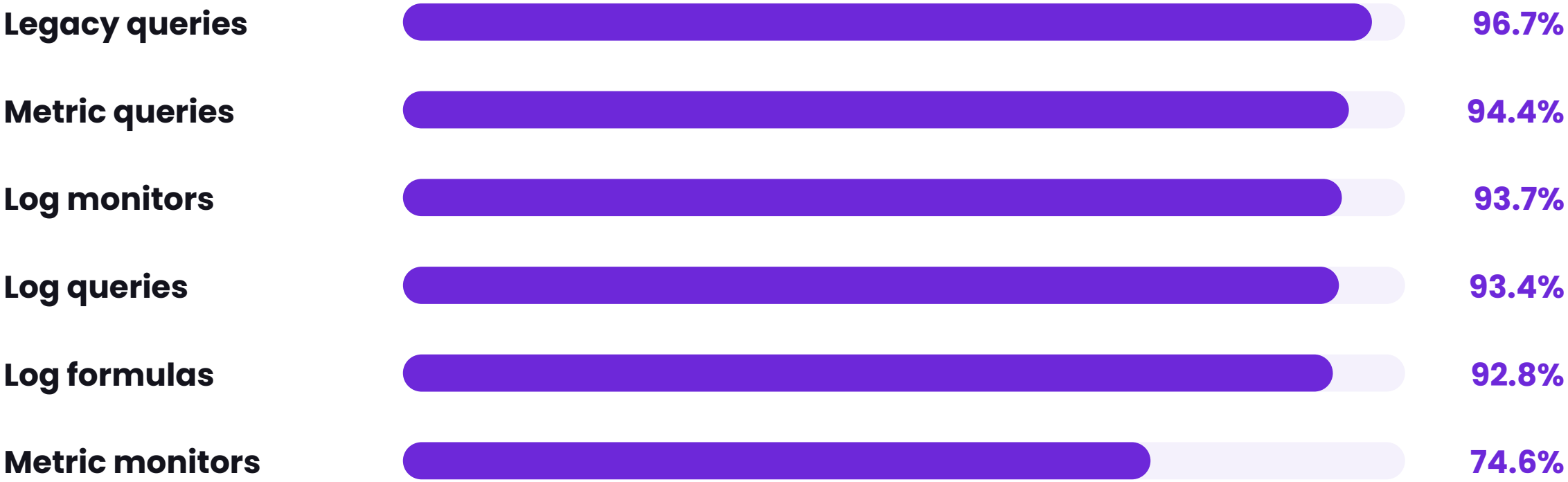
Of 22 items: 13 within 1%, 1 differs, 3 read zero on both, 5 skipped with reasons.

Item	Datadog	Dash0	Difference
Percent used memory	95.053	95.057	**0.004%**
Total allocated memory	225.38M	225.31M	**-0.03%**
Keyspace / total keys / current total	100,160	100,160	**0.00%**
Keys with expiration	164	164	**0.00%**
Hit rate and cache hit rate	100	100	**0.00%**
Fragmentation ratio	0.88143	0.88042	**-0.11%**
Unsaved changes	37.64M	37.51M	**-0.34%**
Commands per second	10,430	10,342	**-0.85%**
Connected clients	**9.14**	**9.71**	**6.2%**

Team spot check on an earlier build. Redis held at about 95% for 9 minutes, compared over a 7-minute window, with the same host feeding both tools. For connected clients, Datadog samples each point and our rule averages each minute.

Backup: Coverage by Query Type

93.7% of 14,172 distinct queries and monitors translate, and 92.2% are exact under a stated condition.



Figures as printed by v0.5.5. 46 metric monitors use anomalies(), and they stay with a person, because we don't guess.